9 - Multiple Access

In this note, we go over topics we took previously (why we have layers, protocols, whatever) to lay the foundation for Aloha and Slotted Aloha. If you understand what the end goal is (one wire/channel, how do multiple people send on it without collision?) you can skip straight to Aloha and other multiple access protocols.

More OSI..

To recap, networks are complex systems made up of many pieces, including hosts, routers, links of various media, applications, and protocols . To organize this structure, network functionality is conceptualized in "layers".

- Each layer implements a specific service via its own internal-layer actions.
- Each layer relies on the services provided by the layer below it.
- Layering helps deal with complex systems because an explicit structure allows for the identification and relationship of the system's pieces.
- Modularization eases maintenance and updating of the system.
- Changing the implementation of a layer's service is transparent to the rest of the system.

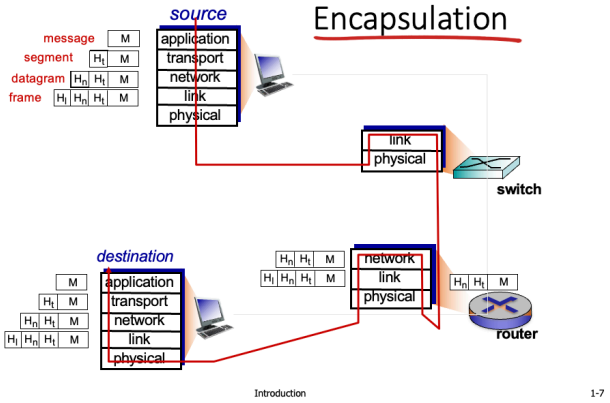
The Internet Protocol Stack

- Application: Supports network applications (e.g., FTP, SMTP, HTTP).
- Transport: Handles process-to-process data transfer (e.g., TCP, UDP).
- Network: Manages the routing of datagrams from source to destination (e.g., IP, routing protocols).
- Link: Manages data transfer between neighboring network elements (e.g., Ethernet, 802.11 WiFi).
- Physical: Transmits raw bits "on the wire".

ISO/OSI Reference Model

- The ISO/OSI model includes two additional layers between the Application and Transport layers .
- Presentation: Allows applications to interpret the meaning of data (e.g., encryption, compression, machine-specific conventions).
- Session: Manages synchronization, checkpointing, and recovery of data exchange.
- The Internet stack is "missing" these layers; if these services are needed, they must be implemented directly within the application.

Encapsulation



- Data moves down the protocol stack from the source, starting as a message (M).
- The Transport layer adds a header (H_t) to create a segment.
- The Network layer adds a header (H_n) to create a datagram.
- The Link layer adds a header (H_l) to create a frame.

Multiple Access Links and Protocols

There are two main types of network links.

- Point-to-point links: Used for dial-up access (PPP) or a link between an Ethernet switch and a host .
- Broadcast links: Rely on a shared wire or medium, such as old-fashioned Ethernet, upstream HFC, or 802.11 wireless LAN .

The Multiple Access Problem

- A single shared broadcast channel can experience two or more simultaneous transmissions by nodes, causing interference.
- A collision occurs if a node receives two or more signals at the exact same time.
- A *multiple access protocol* is a distributed algorithm that determines how nodes share the channel and when they can transmit.
- Communication about channel sharing must use the channel itself, as there is no out-of-band channel for coordination.

Ideal Multiple Access Protocol

- When one node wants to transmit, it can send at the full channel rate of R bps.
- When M nodes want to transmit, each can send at an average rate of R/M.
- The protocol is fully decentralized, meaning there is no special node to coordinate transmissions and no required synchronization of clocks or slots .
- The protocol is simple. I wish life was simple.

Is this possible? not really. We do not live in an ideal world ;c

MAC Protocols

Multiple Access Control (MAC) protocols are divided into three broad classes.

1. Channel Partitioning

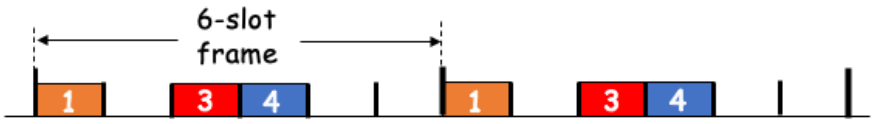
Channel partitioning divides one shared channel into smaller pieces, then gives each user one piece for exclusive use.

The "piece" can be split by:

- time -> TDMA
- frequency -> FDMA
- code -> CDMA

The main advantage is that users interfere less because they are separated in an organized way. The downside is that if one user's assigned piece is not being used, that piece may sit idle instead of being borrowed by others.

TDMA (Time Division Multiple Access)



In TDMA, access to the channel happens in repeating rounds.

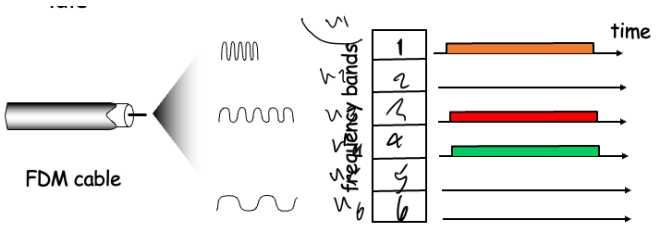
- each station gets one fixed-length time slot in each round
- the slot length is usually about one packet transmission time
- stations take turns using the channel
- if a station has nothing to send, its slot is wasted and goes idle

Example:

- in a `6`-station LAN, suppose only stations `1`, `3`, and `4` have packets
- then slots `2`, `5`, and `6` are empty in that round

> Recall.. TDMA requires usually clock synchronization, it's not decentralised and nodes must agree on a clock

FDMA (Frequency Division Multiple Access)



In FDMA, the total channel spectrum is divided into separate frequency bands.

- each station is assigned one fixed frequency band
- all stations can transmit at the same time
- but each must stay inside its own frequency range
- if a station has nothing to send, its frequency band sits idle
- a central system is usually needed to assign frequency bands to each node

Example:

- in a `6`-station LAN, if only stations `1`, `3`, and `4` have packets
- then frequency bands `2`, `5`, and `6` are unused during that time

CDMA (Code Division Multiple Access)

In CDMA, all users share the same time and same frequency range, but each user is assigned a unique code (also called a chipping sequence).

The cost here is data/computations as Dr. Awad says, instead of time/frequency.

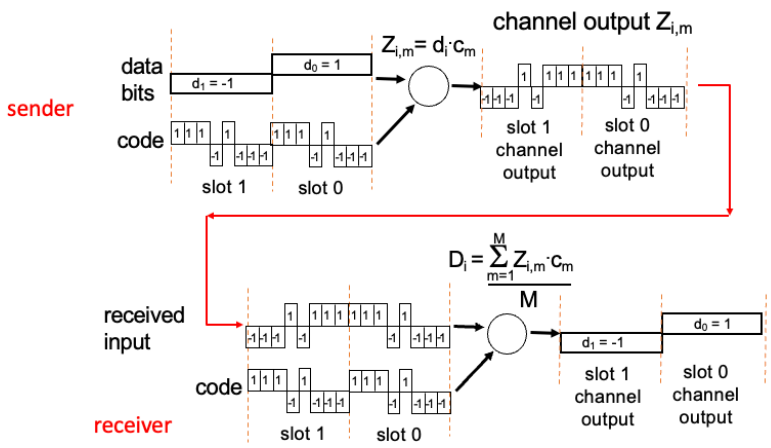
- the code is used to encode that user's data before transmission
- the transmitted/encoded signal is:

$$\text{encoded signal} = (\text{original data}) \times (\text{chipping sequence})$$

- at the receiver, decoding is done using the inner product of the received signal with the same code
- if the users' codes are orthogonal, several users can transmit at the same time with minimal interference

> **Note**
>
> TDMA and FDMA give each user a separate "piece" of the channel directly.
> CDMA: everyone overlaps, but the codes let the receiver pull the users apart.
> Previous set up to distribute codes is usually needed to ensure they are all orthogonal

CDMA Encode / Decode



I recommend focusing on the CDMA note to understand this.

The basic idea is:

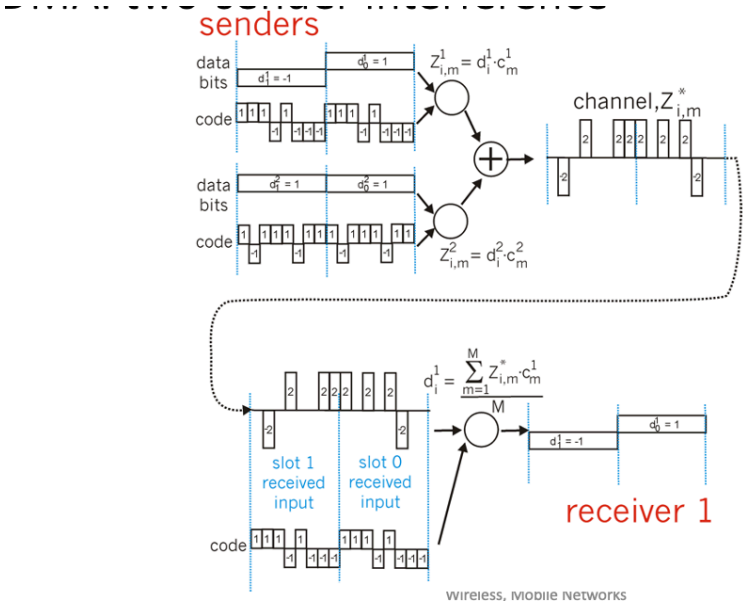
- sender takes one data bit/symbol
- multiplies it by its code sequence
- sends the resulting longer chip sequence into the channel

At the receiver:

- take the received sequence
- multiply / correlate it with the same user's code
- sum the result (inner product)
- this recovers the original data for that user

The code acts like a signature. If the receiver uses the correct signature, the wanted user's signal adds up correctly.

CDMA with Interference



If two senders transmit at the same time, their encoded chip sequences add together in the channel.

The receiver for user `1` still tries to decode using user 1's code.

- the part belonging to user `1` matches and adds up strongly
- the part belonging to the other user cancels out since the codes are orthogonal

CDMA tl;dr: multiple users can coexist on the same channel, and the receiver extracts one user's data by correlating with that user's code.

2. Random Access

In random access protocols, the channel is not pre-divided into fixed pieces.

- any node can try to transmit when it has data
- collisions are therefore possible
- the protocol must define how nodes detect collisions and recover from them, usually by retransmitting later

The benefit is flexibility, especially at low load. The downside is wasted time when collisions happen.

Slotted ALOHA

in the image, 3 packets are tried, and it takes 8 cycles for them to successfully send. $3/8 \approx 0.375$

In Slotted ALOHA:

- all frames are assumed to have the same size
- time is divided into equal-size slots
- nodes are synchronized
- a node may start transmitting only at the beginning of a slot

Coordination requirements:

- needs time synchronization so everyone agrees on slot boundaries
- does not need one central controller deciding who transmits in each slot
- so it is decentralized, but not fully free of coordination

If two or more nodes transmit in the same slot, a collision occurs and the whole slot is wasted.

After a collision:

- the node retransmits in a later slot
- usually with probability p
- this repeats until success

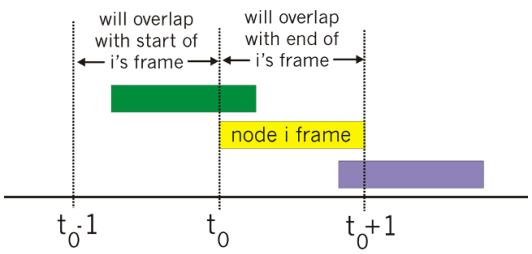
Maximum efficiency:

$$\frac{1}{e} \approx 0.37$$

Slotted ALOHA is simple, but even in the best case only about `37%` of slots carry useful successful transmissions.

Pros	Cons
A single active node can continuously transmit at the full channel rate	Collisions waste whole slots
Highly decentralized: only slot timing needs to be shared	Idle slots are wasted when no one transmits
Simple	Requires clock synchronization
	A collision may be noticed before a full packet time, but the whole slot is still lost

Pure (Unslotted) ALOHA



In Pure ALOHA, there is no slot synchronization.

- a node transmits immediately when a frame arrives
- this makes it simpler than Slotted ALOHA
- but collisions are more likely

Coordination requirements:

- no global clock synchronization is needed
- no central controller is needed

Why more likely?

- if one frame starts at time t_0
- it can collide with any other frame that begins during the vulnerable window around it
- roughly the interval $[t_0 - 1, t_0 + 1]$ in packet-time units

So the vulnerable period is twice as large as in Slotted ALOHA, which makes performance worse.

For one specific node to transmit successfully in Pure ALOHA:

$$\begin{aligned} P(\text{success by given node}) &= P(\text{node transmits}) \\ &\quad \cdot P(\text{no other node transmits in } [p_0 - 1, p_0]) \\ &\quad \cdot P(\text{no other node transmits in } [p_0, p_0 + 1]) \\ &= p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1} \\ &= p(1-p)^{2(N-1)} \end{aligned}$$

What each term means:

- the first `p` = the probability that the chosen node actually transmits
- the first $(1-p)^{N-1}$ = the probability that all the other `N-1` nodes stay silent in the one-packet-time interval before this transmission
- the second $(1-p)^{N-1}$ = the probability that all the other `N-1` nodes stay silent in the one-packet-time interval after this transmission starts

So there are 3 terms because success needs 3 separate conditions to all be true at once:

1. the chosen node transmits
2. nobody else transmits in the vulnerable interval before it

3. nobody else transmits during the period it's transmitting (the vulnerable interval after it)

Maximum efficiency:

$$\frac{1}{2e} \approx 0.18$$

Pure ALOHA is more flexible, but less efficient.

CSMA (Carrier Sense Multiple Access)

tl;dr:

- listen before transmit

That means:

- if the channel sounds idle, transmit
- if the channel sounds busy, wait/defer

Coordination requirements:

- no central controller is required
- no global slot synchronization is required
- each node only needs local carrier sensing, meaning it can listen to whether the medium currently seems busy or idle

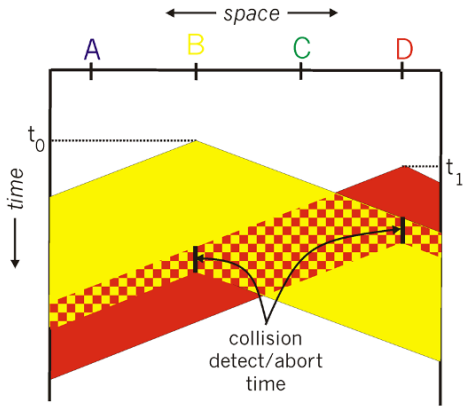
This reduces collisions compared to ALOHA because nodes do not blindly transmit into an already busy channel.

However, collisions can still happen because of propagation delay:

- one node may start transmitting
- another far-away node may not hear it yet.
- so both may decide the channel is idle and transmit nearly at the same time
- specifically noted in the slides spatial layout (distance) causes propagation delays
- on collision, both packets die and entire packet transmission time wasted

CSMA improves efficiency, but it does not eliminate collisions completely.

CSMA/CD (Collision Detection)



CSMA/CD is CSMA plus collision detection.

- a node first listens before transmitting
- while transmitting, it also monitors the channel (unlike pure CSMA)
- if it detects a collision, it stops transmitting early instead of wasting the whole frame time
- instead of both packets dying, for the whole frame, one pulls the plug as quick as possible while the other attempts to recover the non-collided part.

Coordination requirements:

- no central controller is required
- no global clock synchronization is required
- but the hardware must support collision detection while transmitting, which is practical in wired Ethernet and difficult in wireless

This helps because once a collision is detected, the sender aborts quickly and retries later.

Collision detection is practical in wired LANs, where a node can compare the signal it expects with what it actually sees on the medium.

It is difficult in wireless LANs because the node's own transmitted signal is much stronger than the weak signal arriving from another transmitter, so detecting a simultaneous collision is much harder.

> Collision *DETECTION*, not *PREVENTION*.
>
> In CSMA/CD, collisions still can occur. Part of the frame is already corrupted/lost when the detection happens, it just aborts early to ensure not the entire frame slot is lost.
> CSMA/CD: Detection, used in Ethernet, CSMA/CA, Avoidance used in 802.11 (wireless)

3. "Taking Turns" Protocols

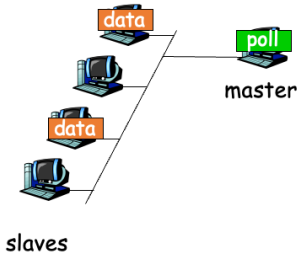
Taking-turns protocols try to combine the good parts of the previous two:

- like channel partitioning, they can be fair and efficient when many users want to send
- like random access, they do not permanently reserve one fixed time/frequency piece for every user

Remember:

- partitioning: low load -> delay in channel access, 1/N bandwidth allocated even if only 1 active node!
- random access: high load -> collisions

Polling



In polling, one special node acts as the master.

- the master asks node `1`: "do you want to send?"
- then node `2`
- then node `3`
- and so on

If a node has data, it transmits when invited. If it has nothing to send, the master moves on to the next node.

This is nice because collisions are avoided: only the node currently being polled is supposed to transmit.

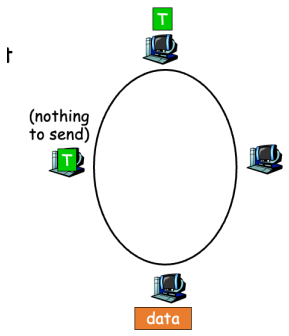
Coordination requirements:

- does need centralization: there is a master/controller
- does not require global slot/clock synchronization like Slotted ALOHA
- all nodes must follow the master's order and wait to be invited

Main downsides:

- polling overhead: the master spends time asking who wants to send
- latency: a node may have to wait a full round before its turn comes
- single point of failure: if the master dies, the whole system is in trouble. oh master, my master!

Token Passing



In token passing, there is no central master asking permission each time.

Instead, a small special control frame called a token moves from one node to the next in a fixed order.

- only the node currently holding the token is allowed to transmit
- after it finishes, it passes the token to the next node
- then that node gets its turn

So again, collisions are avoided because permission to send exists in only one place at a time.

Coordination requirements:

- no central master is required
- no global slot synchronization is required
- nodes must agree on the token order and correctly pass/maintain the token. Max time per machine is also usually restricted.

Main downsides:

- token overhead: time is spent circulating the token itself
- latency: a node may wait for the token to come around
- if the token is lost/corrupted, transmission can stop until the system regenerates it
- if token holder dies, system comes to a halt (fixed by auto-recovery, max time, etc)

> **Note**
>
> Taking-turns protocols are usually better than random access under high load because they avoid repeated collisions. Under low load, their control overhead can feel wasteful compared to simply sending immediately.

Bluetooth, FDDI, IBM Token Ring